

MEMORIA DE PROYECTO TIENDA ALQUEMIA

NATALIA GÁMEZ BAREA

ÍNDICE

1. Introducción	3
2. Implementación.	4
2.1 Uso de funciones contextuales: inyectar menú y cesta	5
3. Metodología de desarrollo	6
4. Modelos de la Base de datos	7
5. Creación de vistas	18
6. Rutas del proyecto	20
6.1 Autenticación	20
6.1.1 Cerrar sesión	20
6.1.2. Envío de correos electrónicos	21
6.1.3. Login y Registro de Usuarios	22
6.1.4 Recuperación de Contraseña (Olvido de Contraseña)	22
6.1.5 Seguridad en la Gestión de Contraseñas	23
6.2 Menú	24
6.3 Carrito	25
6.4 Buscar	27
6.5 Encuentranos	28
6.6 Producto	29
6.7 Catálogo	31
6.8 Admin	32

1. Introducción



Este proyecto consiste en la creación de una tienda online básica en la que se pueden ver productos y organizarlos por categorías , buscar lo que se necesita y añadir artículos a una cesta de compra y comprarlos.

La idea principal fue diseñar una página web funcional que permita a los usuarios navegar de forma fácil y encontrar lo que buscan sin complicaciones. Incluí secciones en el menú principal , el catálogo de productos , la opción de búsqueda , el carrito y una parte para ubicar la tienda o contactarnos.

Durante el desarrollo , se organizaron las rutas del proyecto en distintos archivos para mantener un mejor orden . Se utilizaron librerías como Flask , Flask-Caching y dotenv además de una estructura de base de datos conectada mediante modelos propios con SQLAlchemy . Algunas de las funcionalidades que se implementaron fueron: búsqueda de

productos, carrito de compras, sección de contacto, manejo de usuarios, y administración de categorías y devoluciones.

2. Implementación.

```
import os
from dotenv import load_dotenv
from flask import Flask
from backend.Modelos.database import init_db
from flask_caching import Cache

from routes import *

# Cargar las variables de entorno desde el archivo .env
load_dotenv()

app = Flask(__name__)

# Para el login
app.secret_key = os.urandom(24)

# Configuración del correo utilizando variables de entorno
app.config['MAIL_SERVER'] = os.getenv('MAIL_SERVER')
app.config['MAIL_PORT'] = os.getenv('MAIL_PORT')
app.config['MAIL_USE_TLS'] = os.getenv('MAIL_USE_TLS')
app.config['MAIL_USERNAME'] = os.getenv('MAIL_USERNAME')
app.config['MAIL_PASSWORD'] = os.getenv('MAIL_PASSWORD')
app.config['MAIL_DEFAULT_SENDER'] = os.getenv('MAIL_DEFAULT_SENDER')

# Configuración de caché con Flask-Caching
app.config['CACHE_TYPE'] = 'simple' # Puedes usar 'redis' o 'memcached' si es necesario
cache = Cache(app)

# Inicializar la base de datos
init_db(app)

app.add_url_rule('/', 'home', home)
app.add_url_rule('/busqueda', 'busqueda', busqueda, methods=["GET", "POST"])

app.add_url_rule('/login', 'login', login, methods=["GET", "POST"])

@app.context_processor
def inyectar_menu():
    menu_data = obtener_menu(cache)
    print(menu_data) # Verifica los datos que se están obteniendo
    return dict(menu=obtener_menu(cache))

@app.context_processor
def inyectar_cesta():
    return dict(cesta=obtener_cesta())

# HOME
app.add_url_rule('/', 'home', home)

# AUTENTIFICACIÓN
app.add_url_rule('/login', 'login', login, methods=["GET", "POST"])
app.add_url_rule('/registro', 'registro', registro, methods=["GET", "POST"])
app.add_url_rule('/olvidado_contraseña', 'olvidado_contraseña', olvidado_contraseña, methods=["GET", "POST"])
app.add_url_rule('/restablecer_contraseña/<token>', 'restablecer_contraseña', restablecer_contraseña, methods=["GET", "POST"])

# USER
app.add_url_rule('/informacion_personal', 'informacion_personal', informacion_personal)
app.add_url_rule('/update_usuario', 'update_usuario', update_usuario, methods=["POST"])
app.add_url_rule('/compras', 'compras', compras, methods=["GET", "POST"])
app.add_url_rule('/cerrar_sesion', 'cerrar_sesion', cerrar_sesion, methods=["GET", "POST"])
app.add_url_rule('/cambiar_contraseña', 'cambiar_contraseña', update_contraseña, methods=["GET", "POST"])
```

El archivo principal de la aplicación es app.py donde se configura el servidor , la base de datos , las rutas del sitio y las funcionalidad principales de la tienda online . Se usó Flask como framework base, ya que permite crear aplicaciones web de forma modular y sencilla.

Desde `app.py` se importan las rutas que están organizadas por secciones: por ejemplo, hay rutas para la parte de administración (como productos, devoluciones o categorías), y otras para los usuarios (como el carrito, el inicio de sesión o el catálogo). Esto me ayudó a tener el código más ordenado y fácil de mantener.

También se utilizó un sistema de plantillas HTML para mostrar la información en pantalla, junto con funciones en Python que se encargan de manejar los datos y las acciones del usuario. Por ejemplo:

- Cuando un usuario entra al catálogo, el sistema le muestra todos los productos disponibles.
- Si busca algo específico, el filtro de búsqueda le devuelve solo los productos relacionados.
- Si elige un producto, puede añadirlo a la cesta de compra.
- Desde la cesta puede modificar cantidades o eliminar productos.
- El administrador puede crear, editar o eliminar productos y categorías desde formularios especiales.

Además, configuré el uso de variables de entorno con `dotenv` para guardar información privada, como el correo electrónico del sistema. También integré una memoria caché con `Flask-Caching` para que el sitio cargue más rápido.

Finalmente, añadí rutas para otras páginas importantes como "Nosotros", "Encuétranos" y "Contáctanos", para simular un sitio más completo y funcional.

2.1 Uso de funciones contextuales: inyectar menú y cesta

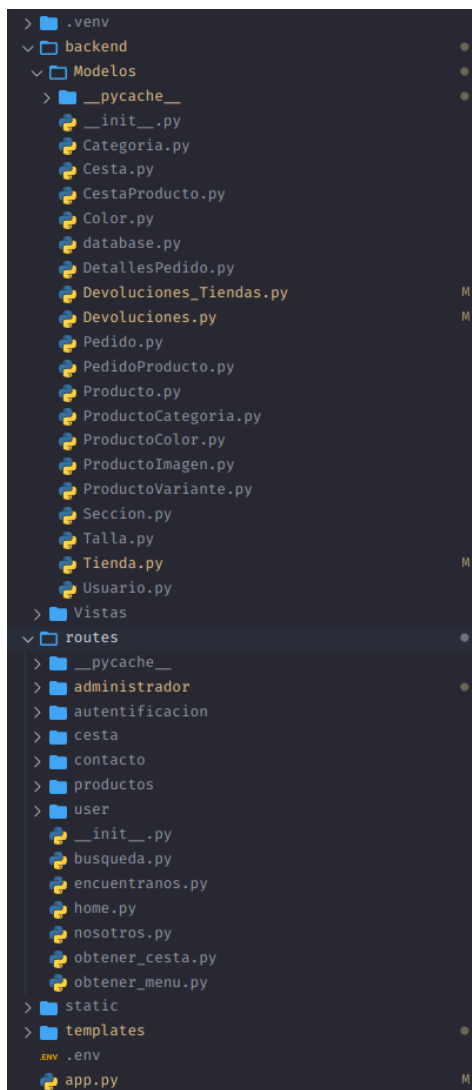
Para evitar tener que repetir código en todas las páginas, utilicé dos funciones contextuales de Flask llamadas `inyectar_menu` e `inyectar_cesta`. Estas funciones se

ejecutan automáticamente en todas las plantillas HTML y permiten que ciertos datos estén siempre disponibles, sin tener que pasarlos manualmente desde cada ruta.

Con `inyectar_menu`, se obtiene el menú de navegación desde la caché, lo que mejora el rendimiento y asegura que siempre se muestran las mismas opciones sin necesidad de repetir el mismo bloque de código. Por otro lado, con `inyectar_cesta` se recuperan los productos añadidos por el usuario, para que la cesta de la compra esté visible en cualquier página.

Esta forma de trabajar hace que el código sea más limpio, reutilizable y fácil de mantener, ya que si hay algún cambio en el menú o en la lógica de la cesta, solo hay que modificarlo en un lugar.

3. Metodología de desarrollo



Para el desarrollo de esta tienda online, seguí una metodología basada en la separación por secciones, dividiendo el proyecto en distintas partes según su funcionalidad. Esta forma de organizar el código me ayudó a trabajar de manera más clara y estructurada, facilitando el mantenimiento y la comprensión del proyecto.

Empecé creando el archivo principal `app.py`, que es donde se configura la aplicación, se conectan las rutas, se inicializa la base de datos y se definen variables importantes. A partir de ahí, separé cada módulo en su carpeta correspondiente, como usuarios, productos, devoluciones y categorías. Esta división me permitió trabajar cada parte por separado, hacer pruebas individuales y evitar confusiones.

Además, reutilicé varias secciones del proyecto, como las plantillas HTML para el diseño del sitio, el menú de navegación y los formularios. Por ejemplo, el mismo menú se muestra en todas las páginas porque lo incluí como

una plantilla compartida. Esto no solo ahorró tiempo, sino que también dio una apariencia más uniforme a toda la aplicación.

Durante el desarrollo, fui probando cada funcionalidad en el navegador y corrigiendo errores cuando algo no funcionaba como esperaba. También utilicé variables de entorno para proteger la configuración del sistema, como los datos del correo.

4. Modelos de la Base de datos

A continuación, se describen los modelos utilizados para estructurar la base de datos de la tienda online. Estos modelos están implementados utilizando SQLAlchemy, un ORM (Object Relational Mapper) que facilita la interacción con la base de datos. Cada modelo corresponde a una tabla en la base de datos y define cómo se deben gestionar los datos.

```
class Categoria(db.Model):  
  
    __tablename__ = 'categorias'  
  
    id_categoria = db.Column(db.Integer, primary_key=True)  
  
    nombre = db.Column(db.String(100), nullable=False)  
  
    productos = db.relationship('Producto', secondary='productos_categorias',  
backref='categorias_relacionadas')
```

```
from backend.Modelos.database import db  
  
from backend.Modelos.CestaProducto import CestaProducto  
  
class Cesta(db.Model):  
  
    __tablename__ = 'cestas'
```

```

    id_cesta = db.Column(db.Integer, primary_key=True, autoincrement=True)

    id_usuario = db.Column(db.Integer, db.ForeignKey('usuarios.id_usuario'), nullable=False,
unique=True)

    fecha_creacion = db.Column(db.DateTime, nullable=False, server_default=db.func.now())

# Relaciones

productos = db.relationship('CestaProducto', backref='cestas', cascade="all, delete-orphan")

```

```

from backend.Modelos.database import db

class CestaProducto(db.Model):

    __tablename__ = 'cestas_productos'

    id_cesta = db.Column(db.Integer, db.ForeignKey('cestas.id_cesta'), primary_key=True,
nullable=False)

    id_variante = db.Column(db.Integer, db.ForeignKey('productos_variantes.id_variante'),
primary_key=True, nullable=False)

    cantidad = db.Column(db.Integer, nullable=False, default=1)

    variante = db.relationship('ProductoVariante', backref='cestas_productos')

```

```

from backend.Modelos.database import db

```



```
class Color(db.Model):

    __tablename__ = 'colores'

    id_color = db.Column(db.Integer, primary_key=True)

    color = db.Column(db.String(50), unique=True, nullable=False)

    img_color = db.Column(db.String(255))

    imagenes = db.relationship('ProductoImagen', back_populates='color')
```

```
from backend.Modelos.database import db

class DetallesPedido(db.Model):

    __tablename__ = 'detalles_pedido'

    id_pedido = db.Column(db.Integer, db.ForeignKey('pedido.id_pedido'), primary_key=True)

    tarjeta = db.Column(db.String(20), nullable=False)

    fecha_caducidad = db.Column(db.Date, nullable=False)

    csv = db.Column(db.String(4), nullable=False)
```

```
from backend.Modelos.database import db
```

```

class DevolucionesTiendas(db.Model):

    __tablename__ = 'devoluciones_tiendas'

    id_devolucion = db.Column(db.Integer, db.ForeignKey('devoluciones.id_devolucion'),
primary_key=True)

    id_tienda = db.Column(db.Integer, db.ForeignKey('tiendas.id_tienda'), primary_key=True)

    # Relación con el modelo 'Devoluciones'

    devolucion = db.relationship('Devoluciones', backref='devoluciones_tiendas_relacionadas',
lazy=True)

    # Relación con el modelo 'Tienda'

    tienda = db.relationship('Tienda', backref='devoluciones_tienda_relacionadas', lazy=True)

```

```

from backend.Modelos.database import db

class Devoluciones(db.Model):

    __tablename__ = 'devoluciones'

    # Define the columns corresponding to the table

    id_devolucion = db.Column(db.Integer, primary_key=True, autoincrement=True)

    descripcion = db.Column(db.Text, nullable=False)

    id_pedido = db.Column(db.Integer, db.ForeignKey('pedidos_productos.id_pedido'),
nullable=False)

```

```

    id_variante = db.Column(db.Integer, db.ForeignKey('pedidos_productos.id_variante'),
nullable=False)

    fecha_devolucion = db.Column(db.DateTime, default=db.func.current_timestamp(), nullable=False)

    hecha = db.Column(db.Boolean, default=False, nullable=False)

    # Relación con el modelo 'DevolucionesTiendas'

    devoluciones_tiendas = db.relationship('DevolucionesTiendas',
backref='devolucion_relacionada', lazy=True , cascade="all, delete-orphan")

```

```

from backend.Modelos.database import db

from sqlalchemy.orm import relationship

class Pedido(db.Model):

    __tablename__ = 'pedidos'

    id_pedido = db.Column(db.Integer, primary_key=True, autoincrement=True)

    nombre_envio = db.Column(db.String(100), nullable=False)

    apellido1_envio = db.Column(db.String(50), nullable=False)

    apellido2_envio = db.Column(db.String(50), nullable=False)

    estado = db.Column(db.String(50))

    tipo_pedido = db.Column(db.String(50))

    fecha = db.Column(db.Date, nullable=False)

    id_usuario = db.Column(db.Integer, db.ForeignKey('usuarios.id_usuario'), nullable=False)

    pedido_productos = relationship('PedidoProducto', backref='pedidos_productos', lazy=True)

```

```
from backend.Modelos.database import db

from sqlalchemy.orm import relationship

from backend.Modelos.ProductoVariante import ProductoVariante

from backend.Modelos.Pedido import Pedido


class PedidoProducto(db.Model):

    __tablename__ = 'pedidos_productos'

    id_pedido = db.Column(db.Integer, db.ForeignKey('pedidos.id_pedido'), primary_key=True)

    id_variante = db.Column(db.Integer, db.ForeignKey('productos_variantes.id_variante'),
primary_key=True)

    cantidad = db.Column(db.Integer, nullable=False, default=1)

    total_producto = db.Column(db.Float, nullable=False, default=0.00)

    pedido = db.relationship('Pedido', backref='pedido_productos_rel', lazy=True)

    producto_variante = db.relationship('ProductoVariante', backref='pedidos_productos')
```

```
from backend.Modelos.database import db
```

```

class Producto(db.Model):

    __tablename__ = 'productos'

    id_producto = db.Column(db.Integer, primary_key=True, autoincrement=True)

    nombre = db.Column(db.String(255), nullable=False)

    precio = db.Column(db.Numeric(10, 2), nullable=False)

    descripcion = db.Column(db.Text, nullable=True)

    id_seccion = db.Column(db.Integer, db.ForeignKey('secciones.id_seccion'), nullable=False) #
Clave foránea a la tabla Secciones

    # Relaciones

    variantes = db.relationship('ProductoVariante', back_populates='producto')

    imagenes = db.relationship('ProductoImagen', back_populates='producto')

    seccion = db.relationship('Seccion', back_populates='productos')

    categorias = db.relationship('Categoria', secondary='productos_categorias',
backref='productos_relacionados')

```

```

from backend.Modelos.database import db

class ProductoCategoria(db.Model):

    __tablename__ = 'productos_categorias'

    id_producto = db.Column(db.Integer, db.ForeignKey('productos.id_producto'), primary_key=True)

```

```
    id_categoria = db.Column(db.Integer, db.ForeignKey('categorias.id_categoria'),
primary_key=True)
```

```
from backend.Modelos.database import db
```

```
class ProductoColor(db.Model):
```

```
    __tablename__ = 'producto_color'
```

```
    id_color_producto = db.Column(db.Integer, primary_key=True, autoincrement=True)
```

```
    id_producto = db.Column(db.Integer, db.ForeignKey('productos.id_producto'), nullable=False)
```

```
    color_id = db.Column(db.Integer, db.ForeignKey('colores.id_color'), nullable=False)
```

```
    img_producto = db.Column(db.Text, nullable=False)
```

```
    producto = db.relationship('Producto', backref='color_images')
```

```
    color = db.relationship('Color', backref='color_images')
```

```
from backend.Modelos.database import db
```

```
class ProductoImagen(db.Model):
```

```
    __tablename__ = 'productos_imagenes_colores'
```

```
id_imagen = db.Column(db.Integer, primary_key=True, autoincrement=True)

id_producto = db.Column(db.Integer, db.ForeignKey('productos.id_producto'), nullable=False)

id_color = db.Column(db.Integer, db.ForeignKey('colores.id_color'), nullable=False)

imagen_url = db.Column(db.String(255), nullable=False)


producto = db.relationship('Producto', back_populates='imagenes')

color = db.relationship('Color', back_populates='imagenes')
```

```
from backend.Modelos.database import db

from backend.Modelos.Color import Color

from backend.Modelos.Talla import Talla


class ProductoVariante(db.Model):

    __tablename__ = 'productos_variantes'


    id_variante = db.Column(db.Integer, primary_key=True, autoincrement=True)

    id_producto = db.Column(db.Integer, db.ForeignKey('productos.id_producto'), nullable=False)

    id_color = db.Column(db.Integer, db.ForeignKey('colores.id_color'), nullable=False)

    id_talla = db.Column(db.Integer, db.ForeignKey('tallas.id_talla'), nullable=False)

    stock = db.Column(db.Integer, default=0, nullable=False)


    # Relaciones

    producto = db.relationship('Producto', back_populates='variantes')
```

```
color = db.relationship('Color', backref='variantes')

talla = db.relationship('Talla', backref='variantes')
```

```
from backend.Modelos.database import db

class Seccion(db.Model):

    __tablename__ = 'secciones'

    id_seccion = db.Column(db.Integer, primary_key=True, autoincrement=True)

    nombre = db.Column(db.Enum('hombre', 'mujer', 'niño', 'niña', 'unisex'), nullable=False)

    # Relación con la tabla Producto (uno a muchos)

    productos = db.relationship('Producto', back_populates='seccion')
```

```
from backend.Modelos.database import db

class Talla(db.Model):

    __tablename__ = 'tallas'

    id_talla = db.Column(db.Integer, primary_key=True, autoincrement=True)

    talla = db.Column(db.String(10), unique=True, nullable=False)
```



```
from backend.Modelos.database import db

class Tienda(db.Model):

    __tablename__ = 'tiendas'

    id_tienda = db.Column(db.Integer, primary_key=True, autoincrement=True)

    pais = db.Column(db.String(100))

    provincia = db.Column(db.String(100))

    ciudad = db.Column(db.String(100))

    codigo_postal = db.Column(db.String(10))

    maps_url = db.Column(db.String(500))

    # Relación con el modelo 'DevolucionesTiendas'

    devoluciones_tiemdas = db.relationship(

        'DevolucionesTiendas',

        backref='tienda_relacionada',

        lazy=True,

        cascade="all, delete-orphan"

    )
```

```
from backend.Modelos.database import db
```

```
from sqlalchemy.orm import relationship

class Usuario(db.Model):

    __tablename__ = 'usuarios'

    id_usuario = db.Column(db.Integer, primary_key=True)

    nombre = db.Column(db.String(50), nullable=False)

    apellido1 = db.Column(db.String(50), nullable=True)

    apellido2 = db.Column(db.String(50), nullable=True)

    email = db.Column(db.String(50), nullable=False)

    contrasena = db.Column(db.String(100), nullable=False)

    is_admin = db.Column(db.Boolean, nullable=False, default = False)

    pedidos = relationship('Pedido', backref='usuarios', lazy=True)
```

5. Creación de vistas

En mi proyecto, he utilizado vistas en la base de datos para simplificar el acceso a las relaciones entre las tablas, como productos, categorías, colores, y más. Las vistas me han permitido definir consultas complejas, lo que ha facilitado la gestión de las relaciones entre las distintas tablas.

Una de las principales ventajas que encontré al utilizar vistas fue la simplificación del código. Al crear vistas que reúnen la información de varias tablas, pude obtener todos los datos necesarios con una sola consulta, eliminando la necesidad de realizar múltiples consultas o combinar joins en el código. Esto ha hecho que el desarrollo sea más ágil y eficiente.

Además, las vistas han mejorado el rendimiento de la aplicación. Al predefinir las consultas más complejas directamente en la base de datos, estas operaciones se ejecutan de manera más rápida y eficiente, ya que la base de datos está optimizada para manejarlas.

Otra ventaja importante es que las vistas me han permitido manejar de manera más sencilla las relaciones complejas entre las tablas. Por ejemplo, en lugar de tener que escribir complicados joins en varias partes del código, ahora puedo acceder a toda la información que necesito de manera centralizada y simplificada.

```
from backend.Modelos.database import db

class VistaProductoCompleto(db.Model):

    __tablename__ = 'vista_productos_completa'

    __table_args__ = {'extend_existing': True} # opcional

    id_variante = db.Column(db.Integer, primary_key=True)

    id_producto = db.Column(db.Integer)

    nombre_producto = db.Column(db.String) # Nombre del producto

    precio = db.Column(db.Numeric(10, 2)) # Precio del producto

    descripcion = db.Column(db.Text) # Descripción del producto

    seccion = db.Column(db.String) # Nombre de la sección

    id_categoria = db.Column(db.Integer) # ID de la categoría

    nombre_categoria = db.Column(db.String) # Nombre de la categoría

    stock = db.Column(db.Integer) # Stock del producto

    color = db.Column(db.String) # Color del producto

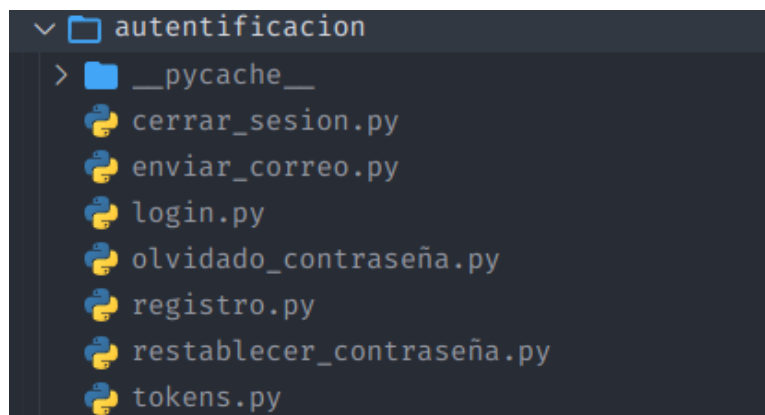
    img_color = db.Column(db.String) # Imagen del color del producto

    talla = db.Column(db.String) # Talla del producto

    imagen_url = db.Column(db.String) # URL de la imagen del producto
```

6. Rutas del proyecto

6.1 Autenticación



He dividido en una carpeta toda la parte de autenticación .

6.1.1 Cerrar sesión

En mi aplicación, implementé la funcionalidad de cierre de sesión para garantizar que los usuarios puedan salir de manera segura. Cuando el usuario decide cerrar su sesión, elimino los datos almacenados en la sesión, como el ID del usuario y su rol (ya sea administrador o no). Esto se logra utilizando la función `session.pop()`, que elimina las claves específicas de la sesión. Luego, redirijo al usuario a la página de inicio de sesión, asegurándome de que no se queden datos residuales de la sesión anterior.

```
from flask import redirect, url_for, session

def cerrar_sesion():

    # cambia
```

```
session.pop("user", None)

session.pop("is_admin", None)

return redirect(url_for("login"))
```

6.1.2. Envío de correos electrónicos

```
def enviar_correo(app, asunto, destinatario, cuerpo, archivos=None):

    mail.init_app(app)

    msg = Message(asunto, recipients=[destinatario])

    msg.body = cuerpo

    if archivos:

        for archivo in archivos:

            with app.open_resource(archivo) as adjunto:

                filename = archivo.split("/")[-1] # Nombre del archivo

                msg.attach(filename, "application/octet-stream", adjunto.read())

    mail.send(msg)
```

Para facilitar la recuperación de contraseñas, incluí un sistema de envío de correos electrónicos. Si un usuario olvida su contraseña, puede solicitar un enlace para restablecerla. Este enlace es enviado a su correo electrónico mediante la integración de Flask-Mail. Utilizo un token seguro para crear un enlace único, que expira después de una hora. Si el enlace ha caducado o es incorrecto, el sistema informa al usuario con un mensaje adecuado. Esta funcionalidad es clave para mejorar la experiencia del usuario, ya que proporciona una forma segura y rápida de recuperar el acceso a la cuenta.

6.1.3. Login y Registro de Usuarios

```
def login():  
  
    # Si el usuarios ya ha iniciado sesión  
  
    if "user" in session:  
  
        if "is_admin" in session and session["is_admin"]:  
  
            return render_template("admin/admin.html")  
  
        return redirect(url_for('informacion_personal'))
```

Para la gestión de usuarios en mi aplicación, he implementado tanto un sistema de inicio de sesión como de registro. Si un usuario ya ha iniciado sesión, se le redirige automáticamente a su página de perfil o al panel de administración, si tiene privilegios de administrador. En caso de que el usuario no esté autenticado, se le presenta el formulario de inicio de sesión.

Cuando un usuario intenta iniciar sesión, verifico que las credenciales coincidan con los datos almacenados en la base de datos. Si las credenciales son correctas, la sesión del usuario se crea y se almacena en la variable de sesión, lo que le permite acceder a las áreas restringidas de la aplicación. Si los datos no coinciden, se le muestra un mensaje de error.

En el caso del registro de nuevos usuarios, los datos ingresados, como nombre, apellido, correo electrónico y contraseña, se validan y se almacenan en la base de datos. Además, las contraseñas son cifradas utilizando la librería bcrypt, lo que garantiza que los datos del usuario estén protegidos.

6.1.4 Recuperación de Contraseña (Olvido de Contraseña)

```

    token = s.dumps(correo, salt='password-reset-salt')

    url_restablecer = f"http://localhost:5000/restablecer_contraseña/{token}"

    cuerpo = f'Haz clic en el siguiente enlace para restablecer tu contraseña: {url_restablecer}'

    enviar_correo(current_app, "Restablecimiento de Contraseña", correo, cuerpo)

    mensaje = "Correo enviado. Revisa tu bandeja de entrada."

    # Que ha tenido exito

    tipo_mensaje = "exito"

else:

    mensaje = "No se encontró una cuenta asociada a ese correo."

    tipo_mensaje = "error"

```

En caso de que un usuario olvide su contraseña, he implementado una funcionalidad para permitirle restablecerla. El usuario ingresa su correo electrónico, y si está registrado, se le envía un correo con un enlace para restablecer su contraseña. Este enlace contiene un token de seguridad, generado a través de un proceso de encriptación. El token es único y tiene una validez de una hora, lo que garantiza que el proceso sea seguro.

Una vez que el usuario hace clic en el enlace y accede a la página de restablecimiento, puede ingresar una nueva contraseña. Al hacerlo, la contraseña se vuelve a cifrar con bcrypt y se actualiza en la base de datos.

6.1.5 Seguridad en la Gestión de Contraseñas

```

clave_secreta = secrets.token_urlsafe(32)

```

```
def obtener_clave_secreta():  
  
    return URLSafeTimedSerializer(clave_secreta)
```

La seguridad de las contraseñas es crucial. Para garantizar que las contraseñas almacenadas en la base de datos sean seguras, utilizo bcrypt para cifrarlas. bcrypt genera un hash único y seguro para cada contraseña, lo que hace casi imposible que alguien pueda descifrar, incluso si accediera a la base de datos.

Además, para la recuperación de contraseñas, utilizo la librería itsdangerous para generar tokens de seguridad que permiten verificar la identidad del usuario que está intentando restablecer su contraseña. Estos tokens son firmados de forma que es casi imposible que se modifiquen sin ser detectados.

6.2 Menú

En mi proyecto desarrollé un sistema de menú dinámico utilizando Flask, el cual se encarga de organizar y mostrar categorías e imágenes de productos de forma eficiente y personalizada. Desde el inicio, mi enfoque fue ofrecer una experiencia de usuario fluida, donde los visitantes pudieran explorar productos por género (hombre o mujer) y visualizar contenido visual atractivo y relevante.

Funcionalidad del Menú

El menú se construye a partir de tres tipos principales de categorías:

- Generales: Todas las categorías existentes en la base de datos.
- Para hombre: Filtradas por la sección "hombre".
- Para mujer: Filtradas por la sección "mujer".

Además, implementé una funcionalidad para seleccionar aleatoriamente tres categorías por género, de modo que el menú principal muestre contenido variado cada vez que se carga. También muestro imágenes aleatorias de productos por sección, lo cual hace el menú más visual y atractivo.

Modelos Utilizados

Me apoyé en los siguientes modelos de mi backend:

- **Categoria:** Define el tipo de producto.
- **Producto:** Contiene los detalles e imágenes de los productos.
- **ProductoCategoria:** Relaciona productos con sus categorías.
- **Seccion:** Permite filtrar productos por género o sección.

Para mejorar el rendimiento y evitar consultas repetidas a la base de datos, utilicé un sistema de caché. Las categorías e imágenes se almacenan temporalmente durante una hora (timeout=3600), lo que reduce la carga del servidor y acelera los tiempos de respuesta del menú.

Gracias a esta lógica, el menú de mi proyecto no solo es funcional, sino también dinámico y escalable. Se adapta al contenido disponible en la base de datos, aprovecha el caché para mayor eficiencia, y mantiene una interfaz actualizada y visualmente atractiva para el usuario.

6.3 Carrito

En el desarrollo de mi proyecto, implementé un sistema de carrito de compras que permite a cada usuario almacenar y visualizar los productos que desea adquirir. Esta funcionalidad es esencial en cualquier tienda en línea, por lo que puse especial énfasis en que fuera eficiente, clara y fácil de mantener.

Gestión de Sesión

El carrito se asocia a cada usuario a través de su sesión (session.get("user")). Si el usuario no está autenticado, el sistema simplemente retorna None, asegurando así que

la funcionalidad del carrito se mantenga restringida a usuarios registrados o activos en sesión.

Recuperación de la Cesta

Para cada usuario, busco si ya existe una cesta (carrito) en la base de datos. En caso de que no haya una creada aún, retorno una estructura vacía con 0 productos y total en cero. Si la cesta existe, consulto todos los productos vinculados a ella mediante el modelo CestaProducto.

Vista Detallada de Productos

Utilicé una vista personalizada llamada VistaProductoCompleto, la cual consolida toda la información necesaria del producto y su variante (como nombre, imagen, talla, color y precio). Esta vista me permite renderizar cada entrada del carrito con todos los datos relevantes, sin tener que hacer múltiples consultas a distintas tablas.

Cálculo de Subtotales y Total

Por cada producto en la cesta, cálculo:

El subtotal individual multiplicando el precio unitario por la cantidad.

El total global del carrito sumando todos los subtotales.

Usé el módulo Decimal para manejar los precios con precisión, y luego convertí a float para facilitar la visualización final.

El resultado de esta función es un diccionario con:

- Una lista detallada de los productos en la cesta.
- El número total de productos.
- El importe total de la compra.

Este diseño me permite integrarlo fácilmente en el frontend, ofreciendo a los usuarios una vista clara, exacta y actualizada de su carrito de compras.

6.4 Buscar

Para facilitar la navegación y permitir a los usuarios encontrar fácilmente productos dentro de la tienda, desarrollé un sistema de búsqueda dinámico e integrado con paginación. Esta funcionalidad permite filtrar productos por nombre utilizando coincidencias parciales y mejora considerablemente la experiencia del usuario.

Entrada del Usuario

El sistema comienza tomando el parámetro búsqueda desde la URL. Este valor representa el término ingresado por el usuario en la barra de búsqueda. Si no se introduce ningún término, simplemente renderizo la plantilla sin realizar ninguna consulta.

Consulta a la Base de Datos

Una vez recibido el término, realizo una búsqueda en la base de datos utilizando SQL con ILIKE, lo que me permite realizar una comparación no sensible a mayúsculas y minúsculas. De esta forma, el sistema encuentra coincidencias parciales en los nombres de los productos, lo cual es útil para capturar errores menores en la escritura o variaciones en los términos de búsqueda.

Paginación

Para manejar resultados extensos, integré una paginación con un límite de 42 productos por página. Determino la página actual a partir del parámetro pagina en la URL y utilizo el método paginate de SQLAlchemy. También calculo el número total de páginas con ceil, asegurándome de que el número sea entero y redondeado hacia arriba.

Manejo de Resultados Vacíos

Si no se encuentran productos que coincidan con la búsqueda, retorno un mensaje claro al usuario indicando que no hubo coincidencias, en lugar de mostrar una página en blanco.

Renderizado de Resultados

Cuando hay coincidencias, paso la lista de productos encontrados junto con el número de página actual, el total de páginas y el término de búsqueda a la plantilla `busqueda.html`. Esto me permite mostrar resultados ordenados, mantener el contexto de la búsqueda y brindar navegación paginada.

6.5 Encuentranos

Como parte de la funcionalidad informativa de mi proyecto, desarrollé una vista llamada "Encuétranos" con el objetivo de mostrar al usuario un listado de todas las tiendas físicas disponibles. Esta sección permite a los visitantes ubicar fácilmente nuestras sucursales filtrando por país, provincia o ciudad.

Consulta de Ubicaciones

Al cargar la página, realizo tres consultas a la base de datos para obtener:

- Todos los países disponibles (`Tienda.pais`)
- Todas las provincias
- Todas las ciudades

Uso `distinct()` para asegurar que los resultados no se repitan, ya que un mismo país o ciudad puede estar asociado a múltiples tiendas. Estos datos me permiten luego ofrecer filtros dinámicos en la interfaz, aunque en esta vista no están implementados directamente los filtros, dejo la estructura preparada.

Listado de Tiendas

También recupero la lista completa de todas las tiendas registradas en la base de datos, sin aplicar filtros iniciales. Esto permite mostrar un mapa o una grilla de todas las sucursales desde el inicio.

Renderizado

Paso todos estos datos a la plantilla `encuentranos.html`, donde se puede mostrar:

- Un desplegable o lista con los países, provincias y ciudades.
- Un listado visual o mapa interactivo con las tiendas.

Este componente cumple una función informativa clave en el proyecto, ayudando al usuario a encontrar puntos de venta físicos, y está diseñado para ser escalable en caso de que más tiendas sean agregadas en el futuro.

6.6 Producto

Para ofrecer una experiencia de usuario completa en la tienda, implementé una vista detallada de producto que muestra información completa de un artículo específico, incluyendo imágenes, variantes por color y talla, y productos recomendados. Esta vista no solo permite al usuario conocer mejor el producto, sino también tomar decisiones de compra más informadas.

Carga del Producto

Cuando se accede a la página de detalle, recupero el producto desde la base de datos utilizando su `id_producto`. Uso `joinedload` de `SQLAlchemy` para cargar en una sola consulta todas las relaciones necesarias:

- Las variantes del producto con su color y talla.
- Las imágenes del producto.
- Las categorías a las que pertenece.

Esto mejora significativamente el rendimiento al evitar múltiples consultas durante el renderizado.

Organización de Imágenes por Color

Creo un diccionario llamado `imagenes_por_color` para organizar todas las imágenes del producto según su color. Esto facilita mostrar únicamente las imágenes relevantes cuando el usuario selecciona un color específico en la interfaz.

Productos Recomendados

Para enriquecer la experiencia de navegación, incluyo hasta 4 productos recomendados. Estos se seleccionan cumpliendo los siguientes criterios:

- No son el mismo producto actual.

- Tienen al menos una imagen.

- Comparten alguna categoría con el producto actual.

Esto ayuda a mantener al usuario explorando opciones similares y potencialmente aumentar el tiempo de permanencia o las ventas cruzadas.

Además, normalizo las imágenes de los productos recomendados:

- Si un producto no tiene imágenes, le asigno placeholders.
- Si solo tiene una imagen, la duplico para mantener la consistencia visual en el frontend.

Selección de Variante (Color y Talla)

El usuario puede seleccionar una combinación de color y talla desde el formulario. Si se reciben estos datos mediante POST, busco la variante correspondiente en la base de datos y paso el `id_variante` a la plantilla. Esto permite manejar correctamente acciones como "Añadir al carrito".

Renderizado Final

Envío todos los datos a la plantilla `producto_detalle.html`, incluyendo:

- La información del producto.
- Las imágenes organizadas por color.
- Los productos recomendados.
- El `id_variante` si se seleccionó una variante válida.

6.7 Catálogo

Para facilitar la exploración de los productos disponibles en la tienda, desarrollé el componente Catálogo, el cual permite listar y filtrar artículos por sección (como hombre o mujer) y por categoría específica. Este módulo es esencial para ofrecer una experiencia de navegación clara y organizada dentro del ecommerce.

Filtros de Navegación

El catálogo acepta dos parámetros desde la URL:

- `seccion`: para filtrar productos por su sección general.
- `categoria`: para aplicar un filtro más específico por categoría.

Ambos filtros son opcionales, pero si se proporciona al menos uno, la consulta se ajusta dinámicamente. Esto me permite construir una vista reutilizable tanto para secciones completas como para categorías concretas.

Construcción de la Consulta

La consulta a la base de datos se construye de forma progresiva:

- Uso joins para unir `Producto` con `Seccion`, `ProductoCategoria` y `Categoria`.
- Aplico los filtros necesarios según los parámetros recibidos.

Utilizo `distinct` para asegurarme de que no haya productos duplicados al cruzar múltiples relaciones.

Paginación

Implementé una paginación con 42 productos por página. Esto mantiene la carga de la página rápida y mejora la organización visual. Uso `paginate()` de `SQLAlchemy` para obtener los productos de la página actual y calcular el número total de páginas con `ceil`.

Carga de Imágenes y Variantes

Después de recuperar los productos de la página actual, me aseguro de que las imágenes y variantes de cada producto estén correctamente cargadas. Aunque esto ocurre de manera implícita, dejo los accesos explícitos (`producto.imagenes`, `producto.variantes`) para garantizar que estén disponibles al renderizar la plantilla.

Renderizado

Paso todos los datos necesarios a la plantilla `catalogo.html`, incluyendo:

- La lista de productos para la página actual.
- El número total de páginas.
- La sección y categoría seleccionadas, si las hay.

6.8 Admin

Creación de Productos

La vista `crear_producto` permite al administrador agregar nuevos productos al sistema. Para ello:

- Implementé la carga dinámica de variantes por color y talla, junto con su respectivo stock.
- Incluí soporte para múltiples imágenes por variante, clasificadas por color.
- Usé la función `guardar_imagen` para almacenar las imágenes de forma segura en el servidor, generando rutas relativas accesibles desde el frontend.
- Validé que al menos una sección y una categoría estén seleccionadas antes de proceder.

Toda la lógica se encapsula dentro de un bloque `try-except` para capturar errores de la base de datos (`SQLAlchemyError`) y aplicar `rollback` en caso de fallo.

Eliminación de Productos

Para garantizar una eliminación limpia de un producto, implementé la función `eliminar_producto`, que:

- Elimina manualmente todas las variantes e imágenes asociadas.
- Elimina las relaciones con categorías, sin borrar las categorías en sí.
- Finalmente elimina el producto principal, aplicando commit solo al final del proceso.

Esto asegura la integridad referencial y evita inconsistencias o datos huérfanos en la base de datos.

Visualización y Gestión en el Panel Admin

Con la función `obtener_productos_html` creé un panel administrativo donde:

Se listan todos los productos con sus variantes, imágenes, secciones, tallas, colores y stock total.

- Los productos se ordenan por precio de forma descendente.
- Se muestran las relaciones con categorías, permitiendo una visión general organizada y detallada del inventario.

También integré una validación para asegurar que solo administradores autenticados puedan acceder a esta sección.

Gestión de Pedidos, Devoluciones y Categorías

Además de las operaciones CRUD sobre productos, también he desarrollado lógicas complementarias para:

- Pedidos: registro de compras, control del stock y seguimiento del estado del pedido.
- Devoluciones: implementación de una funcionalidad que permite al usuario solicitar devoluciones y al admin procesarlas desde el panel.
- Categorías: creación y asignación dinámica de categorías a productos, permitiendo múltiples relaciones y mejorando la navegación en el frontend.